Music Recommender Systems
Eight Methods, One Honest Comparison

Individual Project - ESADE MSc - Recommender Systems (Prof. Marc Torrens)
Dataset: Last.fm HetRec 2011 (implicit feedback)

1. Executive summary

This project builds a music recommender prototype on the Last.fm HetRec 2011 implicit-feedback dataset and compares eight methods - three non-personalised popularity baselines, item-item and user-user collaborative filtering, content-based filtering, matrix factorisation, and a friendship-based social recommender - using both accuracy and beyond-accuracy metrics, averaged over five random splits. The central finding is that personalisation more than doubles baseline accuracy (item-item CF reaches Precision@10 = 0.177 versus 0.069 for the best popularity baseline), but no single method wins on every objective. Accuracy, diversity, novelty, catalogue coverage, popularity bias, and scalability pull in different directions, so the recommended production design is a portfolio rather than one model.

2. Problem and data

The music track uses implicit feedback only: the signal is how many times each user played each artist, with no explicit ratings and therefore no observed dislikes. This shapes every downstream decision - preference must be inferred as confidence, and evaluation is a ranking problem (did we surface the held-out artists?) rather than rating prediction.

- **Scale:** 1,892 users x 17,632 artists, 92,834 interactions.
- **Sparsity:** 99.72% of the user-artist matrix is empty.
- **Side data:** an 11,946-tag vocabulary (9,749 of them actually applied) over 186,479 (user, artist, tag) assignments (used by content-based filtering), and a 25,434-edge user friendship graph (used by the social recommender).

3. Exploratory analysis: know your data

Listening is extremely concentrated. The Gini coefficient of plays-per-artist is 0.893; the top 100 artists account for 43.7% of all listening, while the median artist has just a single listener. This long tail is the empirical basis for the project's thesis: a recommender optimised only for accuracy can lean on a tiny set of famous artists and still look respectable, while ignoring almost the entire catalogue.

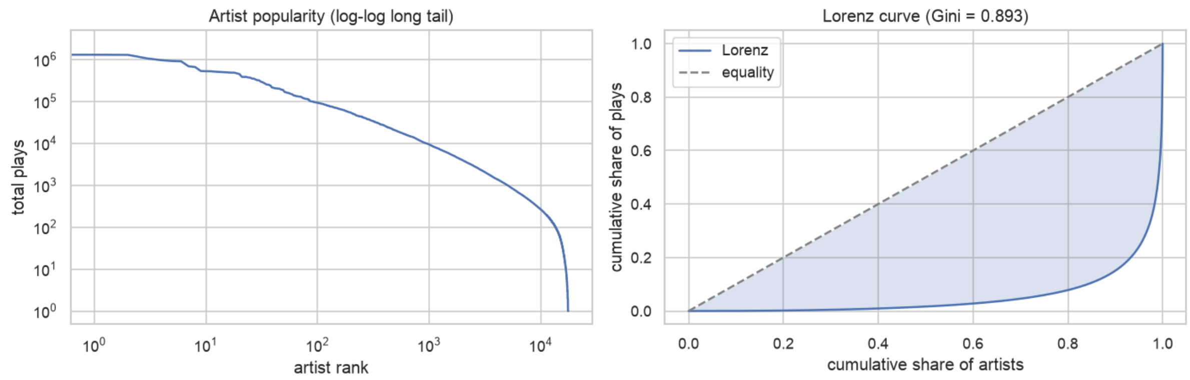

Figure 1. Artist popularity long tail (left) and the Lorenz curve of play concentration (right, Gini = 0.893).

Raw play counts span several orders of magnitude, which motivates log-scaling and confidence weighting before any model sees the data (Figure 2).

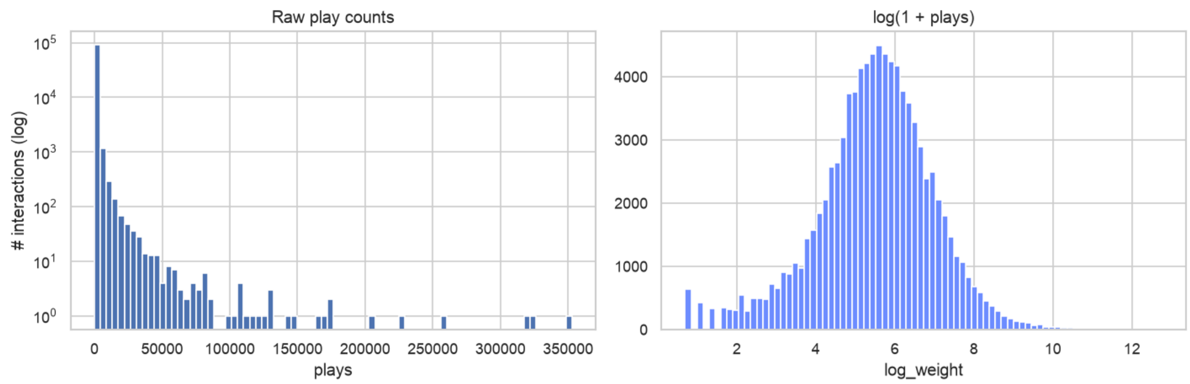

Figure 2. Raw play counts (left) versus log(1 + plays) (right).

User activity, by contrast, is remarkably uniform: HetRec keeps roughly the top 50 artists per user, so the most active users differ little from the median (the EDA notebook lists the top 10). This near-absence of low-activity users is why the cold-start analysis later finds essentially no cold users to stress-test.

Data quality is clean - no duplicate user-artist pairs and no non-positive weights - but the play counts are extremely skewed: the median is 260 plays while the maximum is a single super-fan with 352,698, which reinforces the case for log-scaling before modelling.

3.1 Tags - the content metadata

Tags drive content-based filtering: 186,479 assignments using 9,749 of the 11,946 tags in the vocabulary. They are long-tailed like the listening data, and crucially 5,109 artists (29%) carry no tags at all - a hard ceiling on what content-based filtering can ever reach (Figure 3).

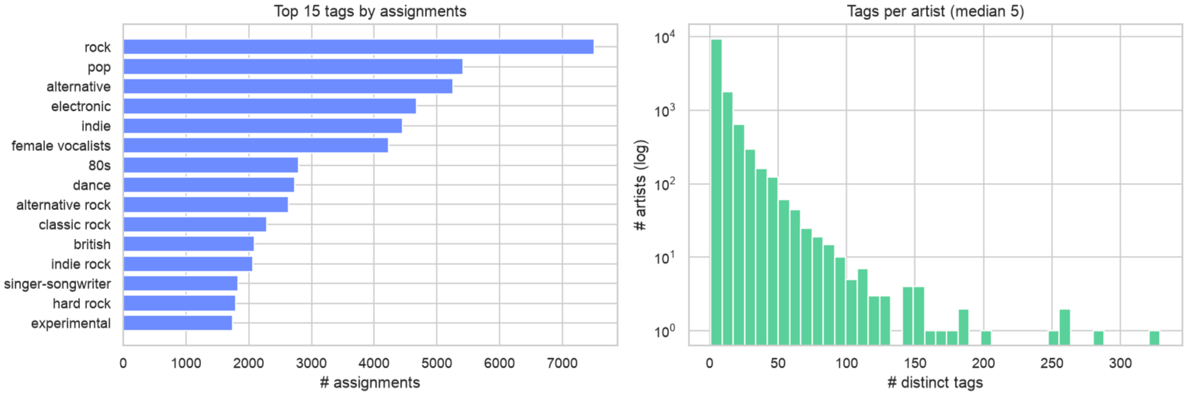

Figure 3. Top tags by assignment count (left) and the distribution of tags per artist (right).

3.2 Friendships - the social signal

The friendship graph is dense (25,434 edges, 13 friends per user on average) and, more importantly, informative: friend pairs share 4.3x more listening overlap than random pairs (mean Jaccard 0.103 versus 0.024). This is the empirical justification for the social recommender added later (Figure 4).

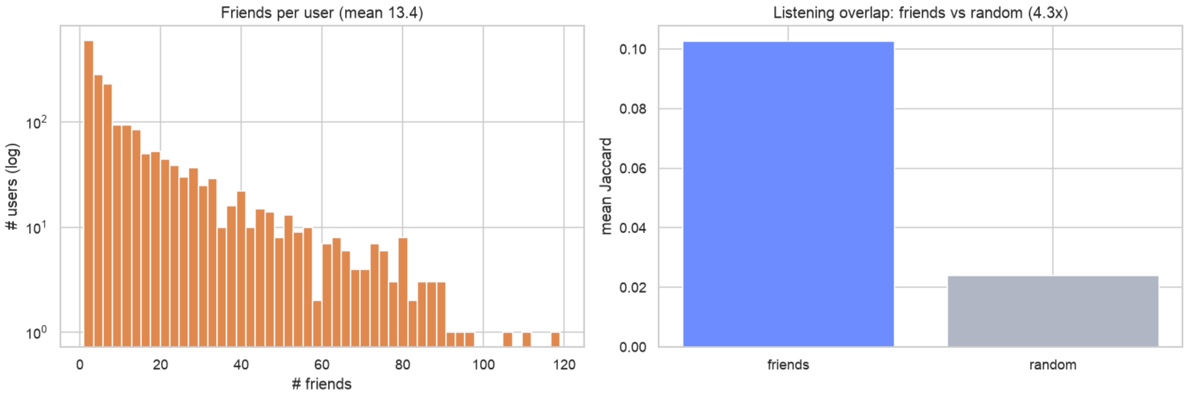

Figure 4. Friends-per-user distribution (left) and friend versus random listening overlap (right).

4. System design and engineering decisions

The codebase is organised as a reusable **src/recsys** package (data, models, eval, utils) with a thin Flask prototype on top and notebooks reserved for exploration. Several deliberate decisions shaped the work:

- **One model interface.** Every recommender implements the same fit / recommend contract, so the evaluation harness and the app treat all methods interchangeably. This is what makes a fair, apples-to-apples comparison possible.
- **One shared split and harness.** A single per-user leave-out split and a single scoring loop are reused by every method; swapping the model keeps everything else constant.
- **Hybrid implementation.** Core algorithms (cosine kNN, TF-IDF profiles, ALS) are hand-implemented to demonstrate understanding, while numpy / scipy / sklearn provide the heavy linear algebra for speed.
- **Test-driven development.** 86 tests at 95% coverage, including closed-form and known-answer checks for the numerically tricky pieces (metrics, the ALS update).
- **Agile, vertical-slice first.** A thin end-to-end path (load -> popularity -> one screen -> one metric) was built before deepening any module, de-risking integration early.

Implicit feedback is handled with the Hu, Koren and Volinsky (2008) confidence formulation, c = 1 + alpha * log(1 + plays): every observed play is a weak-to-strong vote of confidence, sub-linear in the count so that a few superfans cannot dominate.

5. The eight methods and their reasoning

5.1 Popularity baselines (non-personalised)

Three definitions of popular - total plays, distinct listeners, and a damped variant that suppresses mega-hits. These are the comparison floor and the cold-user fallback. Counting distinct listeners beats counting total plays, because total plays let a handful of superfans distort the ranking. Complexity is trivial (a single sorted aggregate).

5.2 Collaborative filtering (item-item and user-user kNN)

Item-item CF scores an artist by its cosine similarity to the artists a user already plays; user-user CF scores by what similar-taste users play. Cosine similarity is hand-implemented as row-normalisation followed by a sparse matrix product, with optional top-k neighbour pruning to bound memory. Item-item is the stronger and more scalable choice under extreme sparsity because artist-to-artist co-occurrence is a more stable signal than user-to-user overlap.

5.3 Content-based filtering (artist tags)

Each artist is represented as a TF-IDF vector over its tags, so distinctive tags outweigh generic ones. A user profile is the play-weighted average of the tag vectors of the artists they play, and recommendations are the nearest artists in tag space. Its structural advantage is cold-start: it can recommend an artist with no listening history at all, which neither CF nor matrix factorisation can do.

An ablation comparing TF-IDF tag weighting against raw L2-normalised tag counts found them essentially equivalent on this data (Precision@10 0.099 vs 0.099; NDCG 0.114 vs 0.115), indicating the tag co-occurrence signal is already informative without inverse-document weighting.

5.4 Matrix factorisation (implicit ALS)

Implicit Alternating Least Squares (Hu, Koren and Volinsky 2008) is implemented from scratch. It learns a short latent factor vector per user and per artist whose dot product reconstructs the confidence-weighted preferences. ALS alternates between two closed-form ridge-regression half-steps (fix items, solve users; fix users, solve items), using the YtY + Yt(C-I)Y trick so the shared term is computed once per iteration. Correctness is verified against a closed-form single-step solution and a synthetic two-cluster dataset, not only against a library. Hyperparameters were chosen by a grid search on a validation slice carved from training (never the test set); the validation winner beat the shipped configuration by only about 2% at twice the fit time, so the cheaper configuration was kept.

5.5 Social recommender (friendship graph)

No other method uses the dataset's friendship graph, so a social recommender was added: a user's score for an artist is the log-scaled amount their friends play it. Structurally it is user-user collaborative filtering with the neighbour set fixed to declared friends rather than taste-similar users - a useful contrast, since social ties and statistical taste similarity are not the same thing.

6. Evaluation methodology

All methods are scored on the same per-user leave-out split across three metric families, and every result is averaged over five random splits to remove single-split fragility; the per-seed standard deviation was only 0.001 to 0.003, so the rankings are stable:

- **Accuracy:** Precision@10, Recall@10, MAP@10, NDCG@10.
- **Beyond-accuracy:** catalogue coverage, intra-list diversity (1 - mean pairwise tag cosine), novelty (mean self-information), and popularity bias (mean recommended popularity plus recommendation exposure Gini).
- **Operational:** training time and per-request serving latency.

7. Results

Personalisation clearly works: every personalised method beats every popularity baseline on accuracy, and item-item CF roughly doubles to triples the baseline across ranking metrics (Figure 5). Notably, the simpler item-item CF beats the more complex ALS on this small, dense dataset - a reminder that sophistication is not automatically valuable.

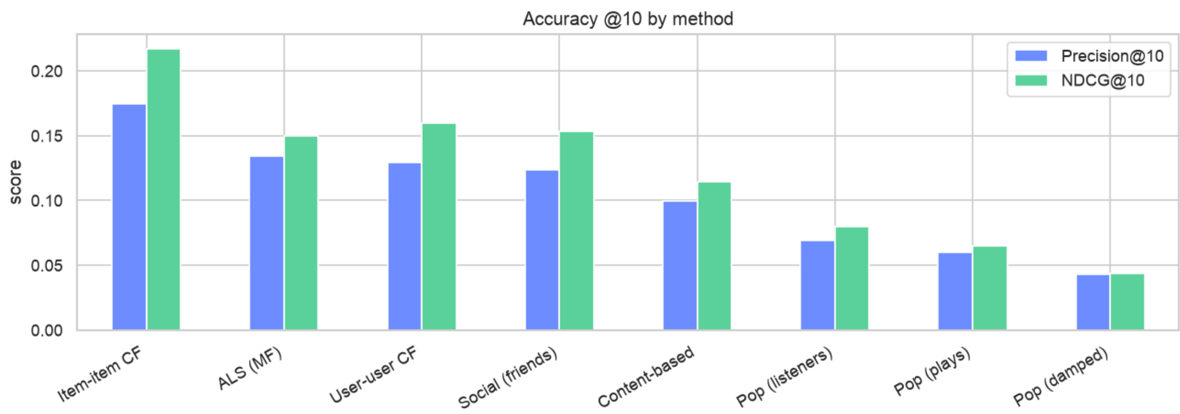

Figure 5. Accuracy at k=10 by method.

But accuracy is not the whole story. Figure 6 shows that no method wins on coverage, diversity, and novelty simultaneously, and Figure 7 makes the accuracy-versus-diversity tension explicit.

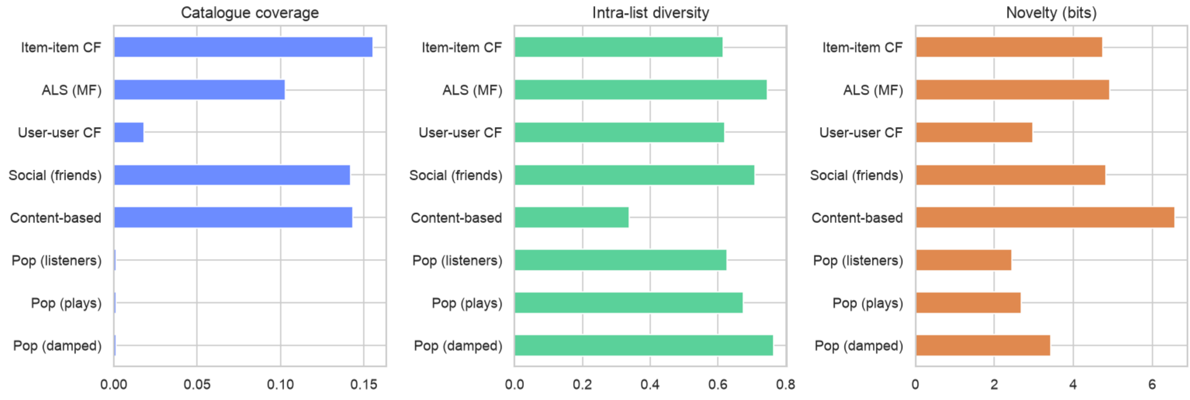

Figure 6. Beyond-accuracy metrics: coverage, diversity, novelty.

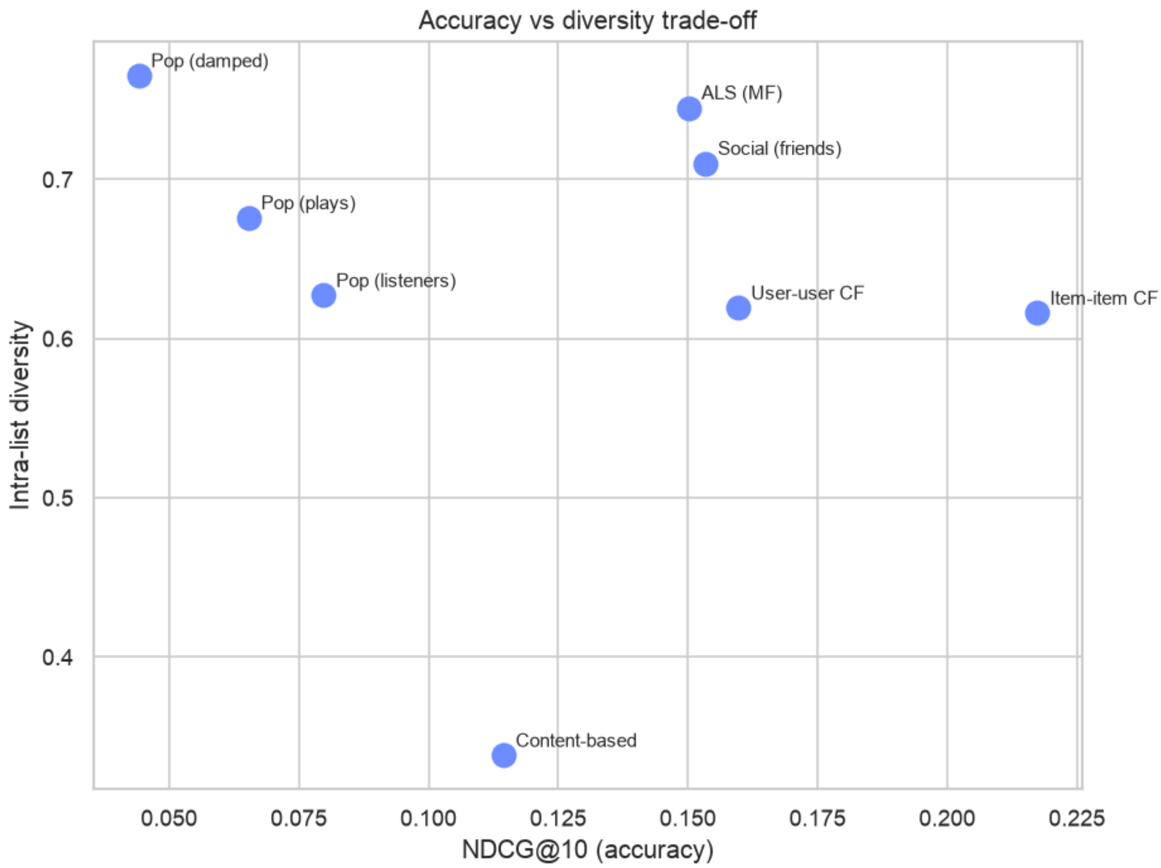

Figure 7. The accuracy versus diversity trade-off.

Popularity bias is measurable and large. Popularity recommenders show the same handful of artists to everyone (catalogue coverage near 0.001 and the highest exposure Gini), while content-based filtering is the least biased toward popular artists. Even user-user CF, though personalised, leans noticeably on crowd-pleasers. Training cost and serving cost also differ sharply by method family (Figure 8).

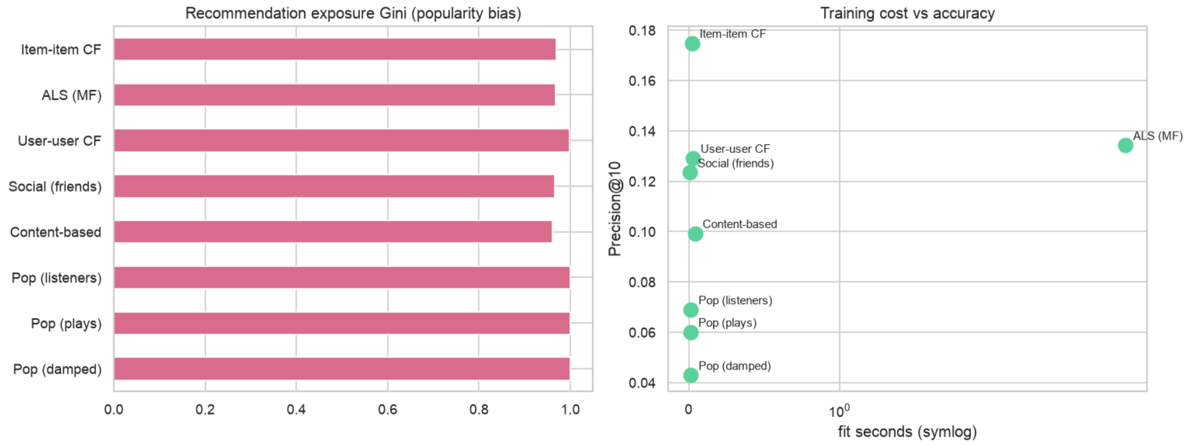

Figure 8. Popularity bias via exposure Gini (left) and training cost versus accuracy (right).

Method	P@10	NDCG	Cov	Div	Nov	PopBias	fit s
Item-item CF	0.177	0.220	0.158	0.616	4.75	0.084	0.02
ALS (MF)	0.134	0.150	0.102	0.746	4.92	0.053	6.48
User-user CF	0.131	0.162	0.019	0.620	2.98	0.140	0.03
Social (friends)	0.123	0.153	0.143	0.712	4.84	0.076	0.01
Content-based	0.101	0.117	0.145	0.337	6.59	0.034	0.04
Pop (listeners)	0.071	0.082	0.001	0.632	2.45	0.185	0.01
Pop (plays)	0.061	0.066	0.001	0.698	2.69	0.164	0.01
Pop (damped)	0.045	0.046	0.001	0.753	3.36	0.126	0.01

Table 1. All methods, mean over 5 random splits (k=10, ~1,884 users; per-seed std was 0.001-0.003, so rankings are stable). Cov = catalogue coverage, Div = intra-list diversity, Nov = novelty (bits), PopBias = mean popularity of recommended artists.

7.1 The social recommender

The friendship-based model is a strong, cheap addition: Precision@10 of 0.123 nearly matches user-user CF (0.131) but with far broader catalogue coverage (0.143 versus 0.018) and higher diversity. In other words, who your friends are is almost as predictive as who is statistically similar to you, and it spreads recommendations across a much wider slice of the catalogue.

7.2 Cold-start

A cold-start analysis exposes a structural divide. Collaborative filtering, ALS, the social model, and popularity can only rank artists seen in training, so of the 2,211 artists with zero training interactions they recommend none; content-based filtering, needing only tags, can recommend 1,227 of those cold artists - its defining advantage. On the user side this dataset is a weak stress test: HetRec keeps roughly the top 50 artists per user, so almost every user has a similar amount of history and accuracy is essentially flat across history quartiles (Figure 9); genuine cold users barely exist here.

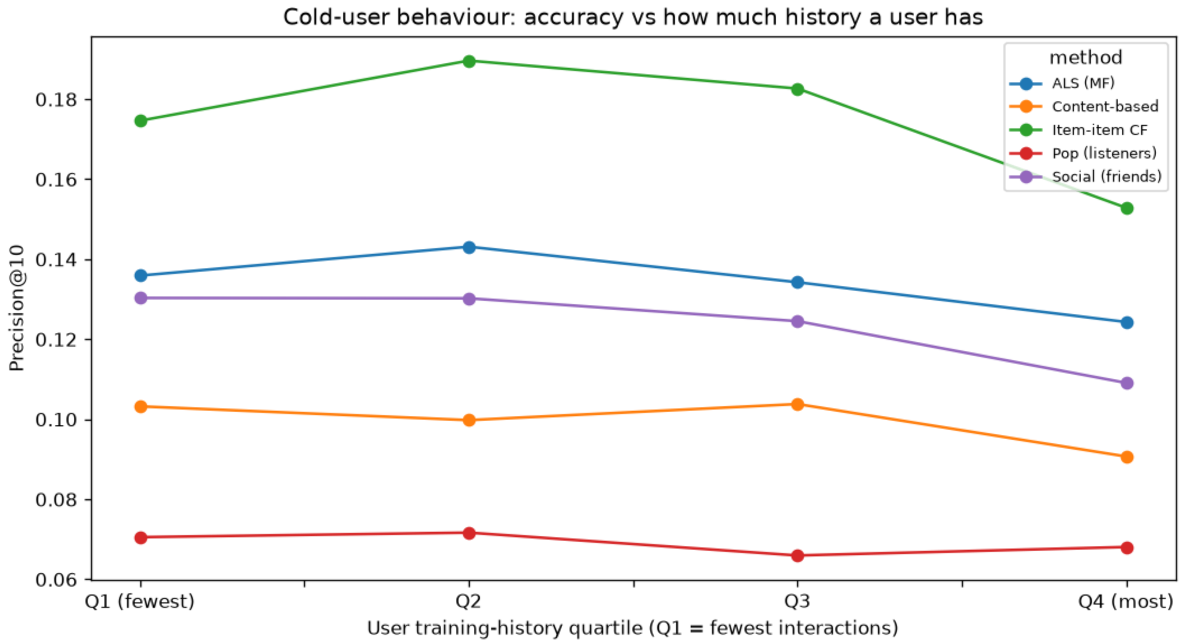

Figure 9. Cold-user accuracy by training-history quartile.

8. Recommendation examples

To show how the algorithms differ in practice, the table below gives the top-5 recommendations for three users from a representative set of methods. Each user's actual most-played artists are listed so the fit can be judged directly.

User 2 - listens to: Duran Duran; Morcheeba; Air

Method	Top-5 recommendations
Popularity (listeners)	Britney Spears; Katy Perry; The Beatles; Rihanna; Avril Lavigne
Item-item CF	Pet Shop Boys; Orchestral Manoeuvres in the Dark; The Human League; Simple Minds; Ultravox
Content-based	Pet Shop Boys; Eurythmics; Moby; Soft Cell; Freur
ALS (MF)	The Prodigy; Pet Shop Boys; Moby; Robbie Williams; Simple Minds
Social (friends)	Pet Shop Boys; Erasure; U2; Thompson Twins; The Cure

User 3 - listens to: Pleq; Segue; Max Richter

Method	Top-5 recommendations
Popularity (listeners)	Lady Gaga; Britney Spears; Katy Perry; The Beatles; Rihanna
Item-item CF	ksandr; Noto; ksandr and I.M.M.U.R.E.; Alva Noto + Ryuichi Sakamoto with Ensemble Modern; Arbre Noir
Content-based	Aphex Twin; #11715; Biosphere; Helios; The Future Sound of London
ALS (MF)	The Cranberries; 30 Seconds to Mars; Muse; Janelle Monáe; Pearl Jam
Social (friends)	Aphex Twin; Venetian Snares; Sigur Rós; Apparat; Björk

User 4 - listens to: Depeche Mode; Porcupine Tree; De/Vision

Method	Top-5 recommendations
Popularity (listeners)	Lady Gaga; Britney Spears; Katy Perry; The Beatles; Rihanna
Item-item CF	Coldplay; Pet Shop Boys; The Beatles; Muse; New Order
Content-based	Muse; The Connells; R.E.M.; Coldplay; No Doubt
ALS (MF)	Muse; Coldplay; 30 Seconds to Mars; Snow Patrol; Hurts
Social (friends)	Dave Gahan; Martin L. Gore; Coldplay; Recoil; And One

The contrast is clearest for the niche user: popularity recommends mainstream pop (Lady Gaga, Britney Spears) to an ambient / modern-classical listener, while the personalised methods correctly surface ambient and IDM artists. Item-item CF stays closest to the exact niche, content-based pulls tag-similar artists, social surfaces what friends play, and ALS generalises - occasionally drifting toward the mainstream for very niche tastes.

9. Critical analysis

- **No universal winner.** Item-item CF leads accuracy and coverage; ALS gives the most diverse of the accurate lists and the lowest popularity bias among strong methods; content-based is the most novel but the least diverse (a tag filter bubble); popularity is a cheap floor that is heavily biased and repetitive.
- **Bias hides inside 'personalised'.** User-user CF has higher popularity bias (0.140) than item-item CF (0.084); personalised does not automatically mean fair.

- **Cold-start, measured.** Of 2,211 artists with no training plays, the collaborative, ALS, social, and popularity methods can recommend none, while content-based can recommend 1,227 of them - a concrete, quantified advantage.
- **Social ties are predictive.** The friendship recommender nearly matches user-user CF on accuracy (0.123 vs 0.131) with far higher coverage and diversity; declared friends are almost as useful as statistical neighbours.
- **Scalability is a trade, not a winner.** Memory-based CF trains in about 0.02s but stores an item-item similarity matrix that grows with the catalogue squared; ALS pays several seconds of training but serves from compact factors in about 0.2ms.
- **Honesty about metrics.** Exposure Gini is high for every method because ten slots across 1,884 users can only ever touch a fraction of 17,632 artists; it is read as a relative concentration measure, and intra-list diversity is only meaningful in tag space.

10. Honest limitations and threats to validity

Several earlier shortcomings were fixed rather than excused: results are now averaged over five splits, ALS was tuned on a validation set, cold-start was measured, and the friendship graph was turned into a working method. The limitations that remain are largely inherent to the dataset and to offline evaluation, and are stated explicitly below.

- **No temporal split is possible.** The interaction file has no timestamps (only aggregate play counts), so a time-based split cannot be built. Evaluation therefore uses a random per-user hold-out, which can leak later listens and likely overstates real-world performance; this is a dataset constraint, not a design choice.
- **Accuracy rewards re-discovery, not discovery.** Relevant items are artists the user already played and we held out, so accuracy favours re-finding known tastes and is structurally biased toward popular items. This is inherent to offline evaluation - measuring genuine discovery needs an online A/B test or user study; the beyond-accuracy metrics soften but cannot remove it.
- **ALS validated by tests, not by a reference library.** Hyperparameters were grid-searched on a validation split, but the hand-written ALS is checked only against a closed-form solution and a synthetic dataset, not the implicit library.
- **Beyond-accuracy blind spots.** Intra-list diversity is computed only in tag space (untagged artists are skipped), novelty is derived from training popularity, and exposure Gini is near-saturated for every method because ten slots across 17,632 artists can only ever touch a fraction of the catalogue.
- **Small, dense dataset.** HetRec keeps roughly the top 50 artists per user, which makes the popularity baseline unusually strong, flatters the neighbourhood methods, and (as the cold-start analysis confirmed) leaves almost no genuine cold users to test; generalisation to a larger, sparser catalogue is untested.
- **Scalability is descriptive only.** Training and serving costs are reported at this dataset's scale; none of the methods were stress-tested at production scale.
- **Prototype UX.** The interface is functional and not user-tested.

11. Conclusion and recommendation

Accuracy is necessary but not sufficient. The right production system for this dataset is a portfolio: ship item-item CF as the default for its strong accuracy, broad coverage, and near-instant training; blend in content-based filtering for cold-start coverage and novelty; use the social recommender to widen coverage where a friendship graph exists; and keep a popularity model as the fallback for brand-new users. Natural next steps are hybrid score blending, a learning-to-rank layer, and a cross-check of the hand-written ALS against the implicit library.

Reproducibility. The dataset is fetched by scripts/download_data.py; scripts/build_processed.py caches the processed artifacts; scripts/evaluate_baselines.py regenerates the multi-seed comparison; scripts/tune_als.py reproduces the ALS validation search; scripts/cold_start.py the cold-start analysis; scripts/recommendation_examples.py the examples; scripts/content_variant.py the TF-IDF ablation; scripts/eda_stats.py the EDA headline numbers; `python main.py` runs the whole pipeline end to end; the notebooks regenerate every figure; and the full test suite (86 tests, 95% coverage) guards correctness.